

Full Stack Summary – Inheritance

Today Covered Topics:

- ❖ Inheritance
- ❖ Types of Inheritance

➤ Inheritance:

When a parent behaviour is inherited to child class is called inheritance. The class is inherited from is called parent class (or) super class (or) base class. The class is inherited is called child class (or) subclass.

Purpose:

- Method Overriding
- Reusability

Advantage:

- Code reusability
- Easy Maintenance
- Extensibility
- Method Overriding
- Organized code.

➤ Types of Inheritance:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

➤ Single Inheritance:

A class inherits from one parent class.

A
↓
B

Eg:

```
class Things {  
    void movies()  
    {  
        System.out.println("Show Movies");  
    }  
}  
  
class Chair extends Things{
```

```

        void tamilmovies()
        {
            System.out.println("Display List");
        }
    }
}

```

```

public class Single {

```

```

    public static void main(String[] args) {
        Chair h =new Chair();
        h.movies();
        h.tamilmovies();
    }
}

```

```

}

```

➤ **Multilevel Inheritance:**

A class inherits from a class, which itself inherits from another class.

```

A
↓
B
↓
C

```

Eg:

```

class Flim {
    void movies()
    {
        System.out.println("Show Movies");
    }
}

class Hollywood extends Flim{
    void tamilmovies()
    {
        System.out.println("Display List");
    }
}

class Kollywood extends Hollywood{
    void kmovies() {
        System.out.println("Display kmovies");
    }
}

public class Multilevel {

```

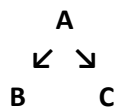
```

        public static void main(String[] args) {
            Kollywood k = new Kollywood();
            k.movies();
            k.tamilmovies();
            k.kmovies();
        }
    }
}

```

➤ **Hierarchical Inheritance:**

Multiple classes inherit from the same parent class.



Eg:

```
package typesofinheritance;
```

```

class Colours {
    void movies() {
        System.out.println("Show Movies");
    }
}

```

```

class Red extends Colours {
    void tamilmovies() {
        System.out.println("Display List");
    }
}

```

```

class Blue extends Colours {
    void kmovies() {
        System.out.println("kmovies");
    }
}

```

```

public class Hierarchical {

    public static void main(String[] args) {
        Red r = new Red();
        Blue b = new Blue();

        r.movies();
        r.tamilmovies();

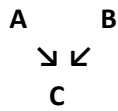
        b.movies();
        b.kmovies();
    }
}

```

```
}
```

➤ **Multiple Inheritance:**

Java does not support multiple inheritance with classes to avoid ambiguity (Diamond Problem). It supports multiple inheritance through interfaces.



Eg:

```
interface A {  
    void display();  
}
```

```
interface B {  
    void show();  
}
```

```
class C implements A, B {  
    public void display() {  
        System.out.println("Display");  
    }  
  
    public void show() {  
        System.out.println("Show");  
    }  
}
```

➤ **Hybrid Inheritance:**

Combination of two or more inheritance types. Java supports it only through interfaces.

Eg:

```
class Movies {  
    void showMovies() {  
        System.out.println("Displaying Movies");  
    }  
}
```

```
interface OTT {  
    void watchOnline();  
}
```

```
class Hollywood extends Movies implements OTT {  
  
    public void watchOnline() {  
        System.out.println("Watching Hollywood Movies Online");  
    }  
  
    void actionMovies() {
```

```
        System.out.println("Displaying Action Movies");
    }
}
```

```
public class HybridInheritance {
    public static void main(String[] args) {

        Hollywood h = new Hollywood();

        h.showMovies();
        h.watchOnline();
        h.actionMovies();
    }
}
```